Permissions — Amaliy mashg'ulot

Install-time · Runtime · Special ruxsatnomalar · Jetpack Compose (10-Ma'ruza — Permissions)

Talaba: Sirojbek Xolmatov | Guruh: AK2024010 | Sana: 30.10.2026

Mashg'ulot ikki qismdan iborat: A — nazariy savollar va vaziyatlar tahlili, B — Android Studio'da bajariladigan amaliy topshiriqlar.

A qism. Nazariy savollar va javoblar

1-savol

Kontekst: Talaba ilovasiga rasmga olish funksiyasini qo'shmoqchi. U birinchi navbatda manifestga CAMERA ruxsatini yozdi va darhol dialog kodini yozishga o'tdi. Jamoadoshi esa: “Avval ruxsatnoma umuman kerakmi-yo'qmi, shuni hal qilish kerak”, deydi.

Vazifa: Ma'ruzadagi ruxsatnomalar bilan ishlash jarayonining (workflow) to'rtta qarorini ketma-ketlikda sanab bering. Ruxsatnomasiz ham bajariladigan holatga misol keltiring va nega bu eng yaxshi variant ekanini tushuntiring.

Javob:
1) Workflow to'rtta qarori ketma-ketligi:
1. Bu funksiya rostdan ham ilovaga kerakmi?
2. Ruxsat olmasdan ham bajarsa boladimi (masalan Intent orqali tizimga bervorish)?
3. Kerak bosa qanaqa ruxsat (install-time yoki runtime)?
4. Foydalanuvchiga to'g'ri tushuntirib vaqtida sorash.

2) Ruxsatnomasiz holat misoli: Profilga rasm qoyish uchun CAMERA ruxsatini soramasdan, tizimning Photo Picker yoki `ACTION_IMAGE_CAPTURE` intentini chaqirish kifoya. Tizim kamerani ochib rasmni ilovaga qaytaradi. Bu eng yaxshi variant chunki ilova xavfli ruxsat sorab user ni chochitmidi, dasturchi ham dialog kodlari yozib bosh qotirmaydi.

2-savol

Kontekst: Jamoadoshingiz Android ruxsatnomalarini “manifestdagi bitta qator” deb hisoblaydi va barcha ruxsatnomalar uchun bir xil kod yozadi.

Vazifa: Ruxsatnomalarning uchta toifasini (install-time, runtime, special) ayting va har biri qanday berilishini tushuntiring. Toifa yozilishi kerak bo'lgan kod hajmini nega belgilashini ko'rsating.

Javob:
1) Uchta toifa:
• *Install-time:* Ilova ornatilayotganda avtomat beriladigan xavfsiz ruxsatlar (masalan internet, bluetooth ulash).
• *Runtime:* Ilova ishlayotgan vaqtda foydalanuvchidan dialog orqali soraladigan xavfli ruxsatlar (kamera, joylashuv, kontaktlar).
• *Special:* Tizim sozlamalariga kirib foydalanuvchi ozi qolda yoqib beradigan maxsus ruxsatlar (overlay, fayllarni toliq boshqarish).
2) Kod hajmi: Install-time uchun manifestga bitta qator yoziladi kod yozish umuman keremas. Runtime da esa ruxsatni tekshirish, dialog chiqarish, rad etilsa rationale korsatish deb kod ancha kopayadi. Special da tizim sozlamalarini intent bilan ochish va qaytganda lifecycle tekshirish kodlari yoziladi.

3-savol

Kontekst: Dasturchi manifestga INTERNET ruxsatini qo'shdi, keyin esa har bir tarmoq so'rovi oldidan checkSelfPermission bilan tekshiruv yozmoqda. Boshqa ilovada esa xuddi shu qator unutilgan.

Vazifa: Normal ruxsatnomalar uchun nega runtime tekshiruv va dialog kerak emasligini tushuntiring. Manifestdagi qator unutilsa, ilova qanday xato (exception) beradi? Ma'ruzadagi normal ruxsatnomalarning uchta misolini keltiring.

> **Javob:**
> Normal ruxsatnomalar foydalanuvchining shaxsiy malumotlariga yoki qurilma xavfsizligiga xavf tugdirmaydi, shuninguchun runtime tekshiruv va dialog keremas, tizim ilova ornatilayotgandayoq ruxsat bervoradi.
> Agar manifestdagi INTERNET qatori unutilsa tarmoq so'rovi ketayotganda ilova `SecurityException: Permission denied (missing INTERNET permission)` berib darhol crash boladi.
> Uchta misol: `ACCESS_NETWORK_STATE`, `CHANGE_WIFI_STATE`, `SET_WALLPAPER`.

4-savol

Kontekst: Bir kompaniyaning ikkita ilovasi bor: “Bank” va “Hamyon”. Hamyon bankning AccountsService xizmatiga ulanmoqchi, xizmat esa uz.ttpu.bank.permission.READ_ACCOUNTS ruxsatnomasi bilan himoyalangan. Uchinchi tomon ilovasi ham xuddi shu ruxsatnomani so'ramoqda.

Vazifa: Signature ruxsatnomasi kimga beriladi? Manifestda uni e'lon qilishda qaysi protectionLevel yoziladi? Hamyon va begona ilova uchun natija nima bo'ladi, va foydalanuvchiga dialog ko'rsatiladimi?

> **Javob:**
> Signature ruxsatnomasi faqat xizmatni yaratgan dasturchining o'sha sertifikati (keystore) bilan imzolangan ilovalargagina beriladi. Manifestda e'lon qilishda `android:protectionLevel="signature"` yoziladi.
> Hamyon va Bank bitta sertifikat bilan imzolangani uchun Hamyon ilovasiga ruxsat avtomatik beriladi va dialog korsatilmidi. Begona ilova esa boshqa kalitda imzolangani uchun tizim unga ruxsat bermaydi va hech qanaqa dialog ham chiqmidi rad boladi.

5-savol

Kontekst: Talaba install-time ruxsatnomalarni ham “ilova ishlayotganda dialog orqali so'rash kerak” deb o'ylaydi.

Vazifa: Install-time ruxsatnomalar foydalanuvchiga qachon va qayerda ko'rsatilishini, tizim ularni qachon berishini tushuntiring. Bu toifaga qaysi ikki turdagi ruxsatnomalar kiradi?

> **Javob:**
> Install-time ruxsatnomalar ilova Google Play do'konidan yuklab olinayotganda korsatiladi va foydalanuvchi o'rnatishni bosishi bilan tizim ularni ilovaga avtomat tarzda beradi.
> Bu toifaga 2 turdagi ruxsatnomalar kiradi: **Normal ruxsatnomalar** (normal permissions) va **Imzo ruxsatnomalari** (signature permissions).

6-savol

Kontekst: Ilova minSdk 21 bilan chiqarilgan. Android 5.1 (API 22) qurilmasida CAMERA uchun dialog chiqmayapti, lekin kamera ishlayapti. Android 14 da esa dialog chiqadi.

Vazifa: Bu farqni tushuntiring: runtime ruxsatnomalar qaysi versiyada joriy qilingan va nega API 22 da runtime turdagi ruxsatnoma o'rnatish vaqtida beriladi? Ma'ruzadagi granted(kind, sdk) modeli bu qoidani qanday ifodalaydi?

> **Javob:**
> Runtime ruxsatnomalar Android 6.0 (API 23 Marshmallow) da joriy qilingan. API 22 da esa eski tizim ishlagani sababli hamma ruxsatnomalar (shu jumladan kamera ham) ilovani ornatish vaqtida berilgan, shuninguchun dialog chiqmasdan ishlayveradi.
> Ma'ruzadagi `granted(kind, sdk)` modelida agar `sdk < 23` bolsa barcha ruxsatnomalar true bop qaytadi, `sdk ⩾ 23` va `kind == RUNTIME` bolgandagina foydalanuvchiga dialog chiqarib soraladi.

7-savol

Kontekst: Foydalanuvchi kamera ruxsatini ikki marta rad etdi. Dasturchi tugma bosilganda yana launch() chaqiradi, lekin ekranda hech narsa ko'rinmaydi va ruxsat darhol rad etilgan holda qaytadi.

Vazifa: Runtime ruxsatnoma holatlarini (hali so'ralmagan → bir marta rad etilgan → ikki marta rad etilgan) tartib bilan yozing. Har holatda shouldShowRationale qiymati qanday bo'ladi va kod nima qilishi kerak? Ikki marta rad etilgandan keyin foydalanuvchini qayerga yo'naltirish kerak?

> **Javob:**
> **Holatlar ketma-ketligi:**
> 1. *Hali so'ralmagan* -> `shouldShowRationale = false`, tizim dialogi chiqadi.
> 2. *Bir marta rad etilgan* -> `shouldShowRationale = true`, nega kerakligini tushuntiruvchi UI (rationale) korsatiladi.
> 3. *Ikki marta rad etilgan* -> `shouldShowRationale = false`, tizim dialogni bloklaydi va launch() darhol rad qilib qaytadi.
> Ikki marta rad etilgandan keyin foydalanuvchini ilova Sozlamalariga (`Settings.ACTION_APPLICATION_DETAILS_SETTINGS`) yo'naltiruvchi tugma chiqarish kerak.

8-savol

Kontekst: Yangi jamoadosh CameraPermissionExample composable'ini o'qimoqda. Unda rememberPermissionState, @OptIn(ExperimentalPermissionsApi::class) va when ichida uchta tarmoq bor.

Vazifa: Uchta tarmoqning (isGranted, shouldShowRationale, qolgan holat) har biri qachon ishlashini va ekranda nima ko'rsatilishini yozing. Nega rememberPermissionState recomposition'dan omon qoladigan holat qaytaradi? ExperimentalPermissionsApi belgisi nimani anglatadi?

> **Javob:**
> **Uchta tarmoq:**
> • `isGranted`: Ruxsat berilgan holat, ekranda kamera UI yoki preview korsatiladi.
> • `shouldShowRationale`: Oldin rad qilingan holat, ekranda kameraning nega kerakligini tushuntiruvchi matn va qayta sorash tugmasi chiqadi.
> • `else (qolgan holat)`: Hali soralmagan yoki butunlay rad etilgan, ruxsat sorash tugmasi korsatiladi.
> `rememberPermissionState` holatni Compose xotirasida saqlab turadi, recomposition bolganda qayta yaratilmasdan omon qoladi. `ExperimentalPermissionsApi` belgisi kutubxona Google tomonidan tajriba tariqasida chiqarilgani va keyinchalik API ozgarishi mumkinligini bildiradi.

9-savol

Kontekst: Jamoa loyihaga qo'shimcha kutubxona qo'shishni xohlamaydi.

Vazifa: Kutubxonasiz so'rovni rememberLauncherForActivityResult va ActivityResultContracts.RequestPermission() bilan qanday yozish mumkinligini tushuntiring. Joriy holatni qaysi chaqiruv bilan o'qiysiz (ContextCompat.checkSelfPermission), rationale'ni qaysi funksiya bilan aniqlaysiz? Bu yondashuvni Accompanist bilan solishtiring.

> **Javob:**
> Kutubxonasiz yondashuvda `rememberLauncherForActivityResult(ActivityResultContracts.RequestPermission()) { isGranted → ... }` yoziladi. Natijada isGranted boolean qiymat qaytadi. Joriy holatni `ContextCompat.checkSelfPermission(context, permission)` orqali tekshiramiz. Rationale holatini esa `ActivityCompat.shouldShowRequestPermissionRationale(activity, permission)` bilan aniqlaymiz.
> Accompanist ga solishtirganda: Accompanist hamma holatni tayyor state qilib beradi lekin tashqi kutubxona talab qiladi; standart API da esa ortiqcha kutubxonasiz toza yoziladi lekin kod sal kopayadi.

10-savol

Kontekst: Xarita ekrani aniq (ACCESS_FINE_LOCATION) va taxminiy (ACCESS_COARSE_LOCATION) joylashuvni birga so'raydi.

Vazifa: RequestMultiplePermissions kontrakti natijani qanday shaklda (Map<String, Boolean>) qaytarishini va uni qanday o'qishni tushuntiring. Nega bu ikki ruxsatnoma uchun foydalanuvchi bitta interfeys ko'radi? Foydalanuvchi ikkalasini ham rad etsa, ilova nima qilishi kerak?

> **Javob:**
> `RequestMultiplePermissions` kontrakti natijani `Map<String, Boolean>` (masalan `{'permission_nomi': true/false}`) shaklida qaytaradi. Uni `it[Manifest.permission.ACCESS_FINE_LOCATION] == true` deb kalit boyicha o'qiymiz.
> Foydalanuvchi bitta interfeys koradi chunki Android 12 dan boshlab tizim foydalanuvchiga bitta dialogda Aniq (Precise) yoki Taxminiy (Approximate) joylashuvni tanlash imkonini bitta qilib chiqaradi.
> Agar foydalanuvchi ikkalasini ham rad etsa, ilova ochibketmasdan joylashuvsiz ishlashi yoki qolda shahar qidirish imkonini berishi kerak.

11-savol

Kontekst: Dasturchi SYSTEM_ALERT_WINDOW uchun oddiy runtime dialogini chiqarishga urinmoqda, lekin dialog umuman chiqmayapti.

Vazifa: Special ruxsatnomalar oddiy runtime ruxsatnomalardan nimasi bilan farq qilishini tushuntiring. Quyidagi jadvalni to'ldiring: har bir ruxsatnoma berilganligini qaysi chaqiruv bilan tekshirasiz?

Ruxsatnoma	Nimaga ruxsat beradi	Tekshirish chaqiruvi
`SYSTEM_ALERT_WINDOW`	Boshqa ilovalar ustidan oyna chizish (Overlay)	`Settings.canDrawOverlays(context)`
`WRITE_SETTINGS`	Tizim sozlamalarini o'zgartirish (yorqinlik va h.k.)	`Settings.System.canWrite(context)`
`MANAGE_EXTERNAL_STORAGE`	Tashqi xotiradagi barcha fayllarni boshqarish	`Environment.isExternalStorageManager()`
`REQUEST_INSTALL_PACKAGES`	Noma'lum manbalardan APK o'rnatishga ruxsat	`context.packageManager.canRequestPackageInstalls()`
`ACCESS_BACKGROUND_LOCATION`	Ilova yopiq turganda ham fonda doimiy joylashuvni olish	`ContextCompat.checkSelfPermission(context, ACCESS_BACKGROUND_LOCATION) == PERMISSION_GRANTED`

> **Tushuntirish:** Special ruxsatnomalar tizim darajasidagi xavfli funksiyalar bolgani uchun oddiy runtime dialog bilan soralmidi, ularni faqat tizim Sozlamalari ekraniga otib foydalanuvchi qolda yoqib berishi kerak.

12-savol

Kontekst: Foydalanuvchi overlay ruxsatini Sozlamalar (Settings) ekranida yoqib, ilovaga qaytdi, lekin ilova hamon “ruxsat yo'q” deb ko'rsatmoqda.

Vazifa: “Tekshir → Sozlamalarni och → qaytganda yana tekshir” andozasini tushuntiring. Nega Sozlamalardan qaytganda kodingizga hech qanday callback kelmaydi? LifecycleEventEffect(Lifecycle.Event.ON_RESUME) yordamida yuqoridagi kodni tuzating.

> **Javob:**
> «Tekshir -> Sozlamalarni och -> qaytganda yana tekshir» andozasi: Sozlamalar ekrani alohida tizim jarayoni bolgani sababli u yerdan qaytganimizda kodingizga onActivityResult ga callback kelmidi. Shu sababli ekranga qaytilganda yana tekshirish lozim.
> **Tuzatilgan kod:**

```
@Composable
fun OverlayScreen() {
    val context = LocalContext.current
    var canDraw by remember {
        mutableStateOf(Settings.canDrawOverlays(context))
    }
    LifecycleEventEffect(Lifecycle.Event.ON_RESUME) {
        canDraw = Settings.canDrawOverlays(context)
    }
    Switch(
        checked = canDraw,
        onCheckedChange = { openOverlaySettings(context) }
    )
}
```

13-savol

Kontekst: Dasturchi ACCESS_FINE_LOCATION va ACCESS_BACKGROUND_LOCATION ni bitta launch chaqiruvida birga so'ramoqda. Fon joylashuvi ishlamayapti.

Vazifa: Ma'ruzadagi ikki bosqichli tartibni tushuntiring: 1-bosqichda nima so'raladi, 2-bosqich qachon va nega faqat birinchisi berilgandan keyin chaqiriladi? Ikkinchi so'rov nega SDK_INT >= Build.VERSION_CODES.Q sharti bilan o'ralgan?

> **Javob:**
> Ikki bosqichli tartib: 1-bosqichda avval old fon joylashuvi (`ACCESS_FINE_LOCATION` yoki `COARSE`) soraladi. 2-bosqichda esa fon joylashuvi (`ACCESS_BACKGROUND_LOCATION`) faqat 1-bosqich ruxsati berilgandagina alohida soraladi. Ikkalasini bitta launch() da birga soralsa tizim hech birini bermaydi va rad etadi.
> Android 10 (API 29 Q) dan oldin fon joylashuvi alohida ruxsatnoma bolmagan, oddiy joylashuv bilan birga ishlayvergan, shuninguchun `SDK_INT ⩾ Build.VERSION_CODES.Q` sharti qoyiladi.

14-savol

Kontekst: Ilova beshta ruxsatnomani birga so'raydi: ikkita joylashuv, ikkita kontakt va bitta kamera. Talaba beshta dialog chiqadi deb kutgan edi, aslida uchtasi chiqdi.

Vazifa: Permission group nima ekanini va tizim dialoglar sonini qanday kamaytirishini tushuntiring. Ma'ruzadagi uchta guruh misolini keltiring. Nega guruh tarkibiga kodingizda tayanmaslik kerak?

> **Javob:**
> Permission group (ruxsatnomalar guruhi) bir-biriga bogliq ruxsatlarni birlashtiradi. Agar guruhdagi bitta ruxsatga allaqachon ruxsat berilgan bolsa, shu guruhdagi keyingi ruxsat soralganda tizim yangi dialog chiqarmasdan avtomat berishi mumkin. Shu orqali dialoglar soni kamayadi.
> Ma'ruzadagi uchta guruh: `CONTACTS`, `LOCATION`, `CAMERA`.
> Lekin guruh tarkibiga tayanmaslik kerak chunki turli Android versiyalarida va ishlab chiqaruvchilarda guruhlash ozgarishi mumkin.

15-savol

Kontekst: Talaba CAMERA uchun dialog kodini yozdi, lekin manifestga <uses-permission> qo'shishni unutdi.

Vazifa: Nima bo'lishini tushuntiring. Install-time va runtime/special ruxsatnomalar uchun e'lon qilish jarayoni qanday farq qiladi? android:maxSdkVersion atributi qanday holatda ishlatiladi (WRITE_EXTERNAL_STORAGE misoli)?

> **Javob:**
> Manifestga <uses-permission> qoyish unutilsa ilova ishga tushganda yoki ruxsat soralmoqchi bolganda SecurityException berib qulab tushadi, yoki tizim hech qanaqa dialog chiqarmasdan natijani darhol rad deb qaytaradi.
> Farqi: Install-time faqat manifestda yoziladi kodda soralmidi. Runtime/special ruxsatlar esa ham manifestda yozilishi ham kodda soralishi shart.
> android:maxSdkVersion atributi eski versiyalarga kerak bolib yangilarida talab qilinmaydigan holatlarda ishlatiladi. Masalan `WRITE_EXTERNAL_STORAGE` Android 10 dan (API 29) boshlab Scoped Storage kelgani uchun `android:maxSdkVersion="28"` qilinadi.

16-savol

Kontekst: Ilova kameradan ixtiyoriy foydalanadi. Do'kon ilovani kamerasiz qurilmalarda (masalan, Android TV) yashirib qo'ydi, jamoa esa ilova hamma qurilmaga o'rnatilishini xohlaydi.

Vazifa: <uses-feature android:name="android.hardware.camera" android:required="..." /> da required ning true va false qiymatlari do'kon filtrlashiga qanday ta'sir qilishini tushuntiring. required="false" bo'lganda ilova kodda nimani majburan tekshirishi kerak (hasSystemFeature)? Ma'ruzadagi visibleOn modelining to'rtta natijasini izohlang.

> **Javob:**
> `required="true"` bolsa Play Store kamerasi yoq qurilmalarga ilovani umuman korsatmidi.
> `required="false"` bolganda esa ilova kamerasiz qurilmalarga (masalan Android TV yoki planshetga) ham o'rnatilaveradi.
> `required="false"` da dasturchi kodda majburan `context.packageManager.hasSystemFeature(PackageManager.FEATURE_CAMERA_ANY)` deb tekshirishi kerak bolmasa kamerasiz qurilmada xato ketadi.
> visibleOn modelining 4 natijasi: qurilmada apparat bor (korinadi va ishlaydi), apparat yoq va required=true (do'konda yashiriladi), apparat yoq va required=false tekshiruvsiz (crash boladi), apparat yoq va required=false tekshiruv bilan (funksiya o'chib xatosiz ishlaydi).

17-savol

Kontekst: Dasturchi kamera holatini bir marta o'qib saqlaydi. Foydalanuvchi “Faqat bu safar” (Only this time) ni tanlagan edi. Ilova fonga o'tib, bir soatdan keyin qaytganda kamera tugmasi faol ko'rinadi, lekin ishlamaydi.

Vazifa: Bir martalik ruxsatnoma (Android 11, API 30) qanday ishlashini va qachon bekor qilinishini tushuntiring. Nega “granted” qiymatini abadiy keshlamaslik kerak? Ma'ruzadagi ON_RESUME yechimi yordamida kodni tuzating.

> **Javob:**
> Bir martalik ruxsatnoma (Android 11, API 30) ilova faol ishlayotgan vaqtgagina beriladi. Foydalanuvchi ilovani tark etsa yoki ilova fonga otib biroz tursa tizim bu ruxsatni avtomatik bekor qiladi.
> Shuninguchun 'granted' qiymatini remember bilan bir marta oqib xotirada qotirib qoyish xato, fonga otib qaytganda eskirib qoladi.
> **Tuzatilgan kod:**
>
> ```
> @Composable
> fun CameraScreen() {
>     val context = LocalContext.current
>     var hasCamera by remember { mutableStateOf(context.hasCamera()) }
>     LifecycleEventEffect(Lifecycle.Event.ON_RESUME) {
>         hasCamera = context.hasCamera()
>     }
> }
> ```

18-savol

Kontekst: Foydalanuvchi ilovani bir necha oy ishlatmagan. Qaytgach, ilgari berilgan ruxsatnoma ishlamayapti. Boshqa tomondan, yangi versiyada ilovaga READ_CONTACTS endi kerak emas.

Vazifa: (1) Auto-reset (Android 11+) nima qilishini va ilova nima qilishi kerakligini yozing; foydalanuvchi uni ilova uchun qanday o'chira oladi (ACTION_AUTO_REVOKE_PERMISSIONS)? (2) Android 13 (API 33) da keraksiz runtime ruxsatnomalarni qaysi funksiya bilan olib tashlash mumkin (revokeSelfPermissionOnKill) va bu foydalanuvchiga nima uchun foydali?

> **Javob:**
> **(1) Auto-reset (Android 11+):** Agar foydalanuvchi ilovani bir necha oy ochmasa tizim berilgan barcha runtime ruxsatlarni avtomat bekor qiladi. Ilova ruxsat har doim bor deb hisoblamasdan har safar oldindan tekshirishi kerak. Foydalanuvchi ilova sozlamalarida `Intent(Settings.ACTION_AUTO_REVOKE_PERMISSIONS)` orqali buni ochirib qoyishi mumkin.
> **(2) Android 13:** Ilovaga ortiqcha kerak bolmagan runtime ruxsatni `revokeSelfPermissionOnKill(permission)` funksiyasi bilan bekor qilish mumkin. Bu foydalanuvchida ishonch uyg'otadi va ilovaning keraksiz xavfli ruxsatlarni ushlab turmasligini taminlaydi.

19-savol

Kontekst: Sizdan “ruxsatnomalar Android versiyalari bo'yicha qanday o'zgargan” degan qisqa jadval tuzish so'ralmoqda.

Vazifa: API 23, API 30 va API 33 uchun asosiy o'zgarishni bir jumlada yozing. POST_NOTIFICATIONS Android 13 da qanday ruxsatnomaga aylanganini va nega uni API 33 dan past versiyalarda so'ramaslik kerakligini tushuntiring.

> **Javob:**
> **Asosiy o'zgarishlar:**
> • **API 23 (Android 6.0):** Runtime ruxsatnomalar tizimi ilk bor joriy qilingan.
> • **API 30 (Android 11):** Bir martalik ruxsat ("Only this time") va ishlatilmagan ilovalar uchun auto-reset tizimi kiritilgan.
> • **API 33 (Android 13):** POST_NOTIFICATIONS runtime ruxsatga aylandi va revokeSelfPermissionOnKill qo'shildi.
> `POST_NOTIFICATIONS` Android 13 da runtime ruxsat boldi, oldingi versiyalarda bildirishnomalar sukut boyicha ruxsat berilgan edi. Shuninguchun API 33 dan past versiyalarda uni so'rash shartmas va so'ramaslik kerak.

B qism. Amaliy topshiriqlar haqida hisobot

Barcha 6 ta amaliy topshiriq `uz.ttpu.permissionslab` paketida Android Studio loyihasi sifatida to'liq bajarildi va alohida semantik commitlar bilan taqdim etildi:

- **1-topshiriq:** `PermissionModel.kt` — Denied twice modeli (granted, rationale, dialogShows) va ketma-ket 4 ta so'rov sinovi.
- **2-topshiriq:** `ConnectivityScreen.kt` — Normal ruxsatnomalar (INTERNET, ACCESS_NETWORK_STATE) va tarmoq online/offline holati.
- **3-topshiriq:** `CameraScreen.kt` — Kamera ruxsatining 3 holati (berilgan, rationale, rad etilgan -> Sozlamalarni ochish tugmasi).
- **4-topshiriq:** `OverlayScreen.kt` — Special ruxsatnoma (SYSTEM_ALERT_WINDOW), Switch va LifecycleEventEffect(ON_RESUME) da yangilanish.
- **5-topshiriq:** `LocationScreen.kt` — Bir martalik joylashuv (FINE + COARSE) RequestMultiplePermissions bilan va ON_RESUME tekshiruvi.
- **6-topshiriq (Bonus):** `NotificationScreen.kt` — POST_NOTIFICATIONS ruxsati (API 33+) va test bildirishnomasi.